Chapter 7 Nearest Neighbors

- $\arg\max_j P(Y=j \mid X=x_0) \rightarrow$ Optimal (Bayes) decision rule assigns class j with the highest probability.

- Nearest Neighbor allows the probabilities to be estimated locally

  ↳ $P(Y=j \mid X=x_0) = \frac{k_j}{k}$, k_j = number of the k-nearest neighbors from class j and $k = \sum_{j=1}^{G} k_j$,

  where G is the number of classes

  $P(Y=j \mid X=x_0) = \frac{1}{k} \sum_{i \in N_0} I(y_i = j)$, N_0 = set of k-points, I = indicator function

  ↳ $I(y_i = j) = 1$ if i-th neighbor belongs to class j, otherwise 0.

  $\sum_{i \in N_0} I(y_i = j)$ counts how many of the k-nearest neighbors belong to class j

  $\frac{1}{k} \sum_{i \in N_0} I(y_i = j)$ shows the fraction of k-nearest neighbors whose class is j.

- No training is required, this is a memory-based algorithm.

- Also accomodates non-linear boundaries, whereas logistic regression only has a linear decision boundary.

- Bayes classifier: The theoretically optimal classifier that assigns each input to the class with the highest true conditional probability given the features. Thus $[\hat{Y}_{Bayes}(x) = \arg\max_j P(Y=j \mid X=x)]$ But theoretically the probabilities are not known

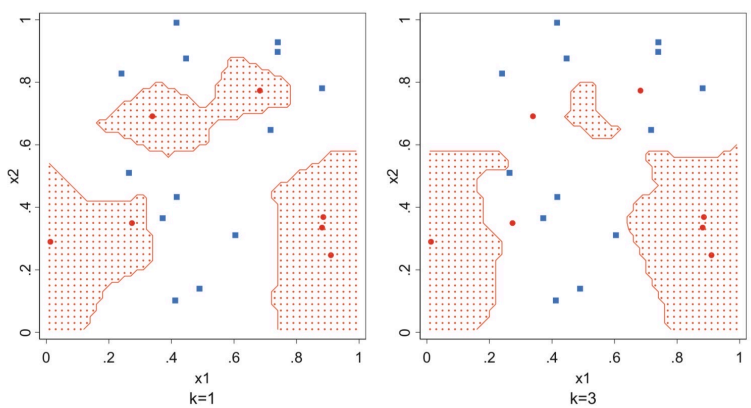

Fig. 7.1 Simulated nearest neighbor example using Euclidean distance with $k=1$ and $k=3$. The blue and red points show the training data. Any new observation falling into the red region would be predicted to be red

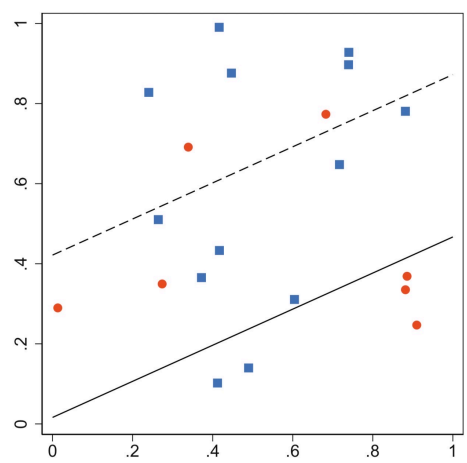

Fig. 7.2 Linear decision boundary based on logistic regression with thresholds 0.5 (solid line) and 0.4 (dashed line)

- Conditional probability / Bayes classifier can only be known when the true underlying model is known

  ↳ KNN approximates this by suggesting to approximate the conditional probability for a given class with the fraction of observations in a local neighborhood of k observations that correspond to the class

  ↳ As the size of training data n increases, asymptotically the k-nearest neighbor classification error for $k=1$ is at most twice the Bayes error.

- Size of neighborhood affects the bias / variance tradeoff.

↳ $k=1$ → Too local and has low bias and high variance

Increasing k makes the estimate more stable by decreasing the variance and increasing bias (Increase distance of k-training obs to x_0)

↳ Initially, this lowers test error but increase train errors, then reverses.

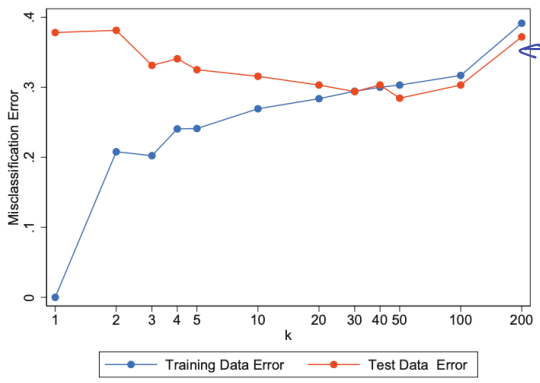

Fig. 7.4 2D simulation: train and test misclassification error as a function of k

← Familiar U-shape on test data

- Reasons for scaling: to avoid the case where some variables dominate determining the nearest neighbors
- Scaling leads to different set of nearest neighbors and a different classification.
- Without scaling, variables that have great ranges have a larger influence on classification
- Another option: perform Mahalanobis transformation of a continous variable before computing similarities or dissimilarities
  ↳ Takes into account the correlation among dimensions

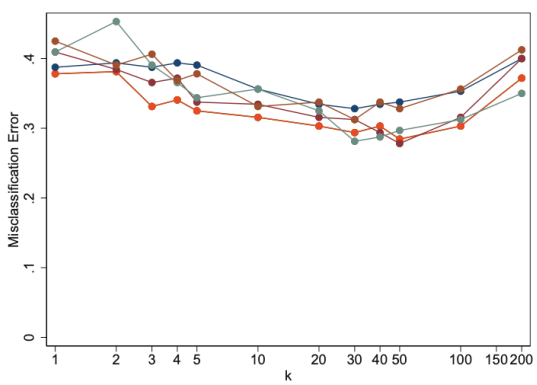

Fig. 7.5 2D simulation: test error curves for different random splits into training and test data. The variability of the curves shows that the best k will also vary by curve

Prediction

- Predicted by majority vote of k local observations
- When ties happen:
  - → Break ties at random
  - → Increase k until the tie is broken
  - → Use the single nearest neighbor (1NN) as a tie breaker

Distance and Similarity

- Typically, Euclidean distance is used (But X-variables must be standardized)
- But often it is more convenient to specify a similarity measure

- When features are all binary, use the Jaccard similarity measure

  ↳ smallest distance = greatest similarity

$$S(x_0, x) = \frac{1 - d(x_0, x)}{d}$$, d_{max} is the largest distance observed in data

Cosine similarity (x_0, x) is used in text mining / document retrieval because X-matrix is very sparse.

↳ Also used in search engines on web.

Also called angular similarity

- Jaccard similarity is the intersection divided by union of two texts

  ↳ If the text is a set of unigrams, the Jaccard metric is the proportion of words used in both texts

$$S_{Jaccard(i,j)} = \frac{TP}{TP + FN + FI}$$

ex

$$x_0 = (1, \overset{FP}{1}, 0, 0, 0, 0)^T \quad \& \quad x_{train} = (\overset{TP}{1}, 0, 0, 0, 0, 0)^T$$

$$S_{Jaccard} = \frac{1}{1+0+1} = 0.5$$

TP = 1
FN = 0
FP = 1

No TN → ignores words in neither of two sets

- Nearest neighbor with a caliper → KNN variation for when the observations are sparse

- Nearest neighbor is good when:

  ↳ Boundary is highly irregular

  ↳ Training data is sufficiently large

  ↳ Highly unstructured → not good for how X-variables relate to outcomes